

GPU Accelerated Approximate Bayesian Computation: A Dual Workload Sequential Baseline

Tejal, Dhyan, Vinay, Arya
CS F422 Parallel Computing, BITS Goa
f20240558@goa.bits-pilani.ac.in

Abstract—Approximate Bayesian Computation (ABC) enables parameter inference when a likelihood function is unavailable, but its repeated forward simulations can be computationally very expensive. We wrote a single threaded C++ ABC rejection sampler and checked it on two models that do not cost the same to run. The Gaussian case has a known posterior so we can tell if the sampler is right. The discrete time stochastic Susceptible–Infected–Recovered (SIR) model is slower and some particles finish earlier than others if we stop when $I = 0$. We also run SIR for a fixed T . Particle counts are 10^4 , 10^5 , and 10^6 . The timings here are what we will compare against once the CUDA version exists.

Index Terms—Approximate Bayesian Computation, simulation-based inference, CUDA, parallel computing, SIR epidemic model, performance baseline.

I. INTRODUCTION

Approximate Bayesian Computation (ABC) is used when the likelihood is hard or impossible to write down. Instead of evaluating the likelihood, ABC draws parameters from a prior, runs a simulator, and keeps the parameters whose simulated output is close enough to the observed data [1]. It shows up a lot in genetics, epidemiology, and ecology, where you have a simulator but not a closed form likelihood [2].

Particles do not depend on each other, so the loop over N can be split across GPU threads. Kulkarni et al. ran batched ABC for epidemiology models on accelerators [3]. ABCpy looked at parallel ABC from an HPC angle, including load imbalance when simulations do not all cost the same [4].

A CUDA number is not useful unless the sequential code is right and the simulator is heavy enough that launch overhead is not the whole story. So this milestone is a single threaded C++ rejection sampler on:

- a Gaussian model with a known posterior;
- a discrete time stochastic SIR model, stopping early when $I = 0$ or always running T steps.

Milestone 1 is the CPU code and the numbers in the tables. CUDA is next.

The CPU RNG is xoshiro256** with Box–Muller. On the GPU we will probably use cuRAND or Philox, so the accepted lists will not be the same draws. Mean, variance, and acceptance rate are what we will match.

II. RELATED WORK

Beaumont et al. [1] introduced ABC for population genetics. Sisson et al. [2] cover the usual variants and how people pick the tolerance.

Kulkarni et al. [3] parallelized the sample, simulate, and distance stages of batched ABC on accelerators. We are following that setup in CUDA later. ABCpy [4] used MPI and noted that some particles take longer than others.

In a stochastic SIR run, some epidemics die out early and take fewer steps. On a GPU that shows up as thread divergence. We timed both early exit and fixed T on the CPU so those two effects are not mixed together later.

III. PROBLEM DEFINITION

A. ABC Rejection Sampling

We have observed data y_{obs} , a prior $\pi(\theta)$, a simulator $f(\theta) \rightarrow y_{\text{sim}}$, summaries $S(\cdot)$, a distance $d(\cdot, \cdot)$, and a tolerance $\varepsilon > 0$. Each particle does:

- 1) Sample $\theta^* \sim \pi(\theta)$.
- 2) Simulate $y_{\text{sim}} = f(\theta^*)$.
- 3) Compute $s_{\text{sim}} = S(y_{\text{sim}})$.
- 4) Accept θ^* if $d(s_{\text{sim}}, s_{\text{obs}}) \leq \varepsilon$.

Accepted θ values come from $\pi(\theta \mid d(S(f(\theta)), s_{\text{obs}}) \leq \varepsilon)$. Sisson et al. note that this target approaches the true posterior when ε is taken to zero [2]. With a nonzero ε we only expect the mean and variance to land near the exact values.

B. Workload 1: Gaussian Inference Model

For Gaussian ABC we can compare against a closed form posterior.

- **Prior:** $\theta \sim \mathcal{N}(0, 1)$.
- **Simulator:** $y_{\text{sim}} \sim \mathcal{N}(\theta, 1)$.
- **Observed value:** $y_{\text{obs}} = 1.0$.
- **Summary statistic:** $S(y) = y$.
- **Distance:** $d = |y_{\text{sim}} - y_{\text{obs}}|$.
- **Tolerance:** $\varepsilon = 0.1$.

At zero tolerance the exact posterior is

$$\theta \mid y_{\text{obs}} = 1 \sim \mathcal{N}\left(\frac{1}{2}, \frac{1}{2}\right). \quad (1)$$

Each particle is two random draws and one comparison. Table III is the same model at $N = 10^5$ for a few ε values.

C. Workload 2: Stochastic SIR Epidemic Model

SIR does more work per particle. If the epidemic dies out, early exit also makes some particles shorter than others.

1) Model Parameters:

- **Parameters:** infection rate β and recovery rate γ .
- **Priors:** $\beta \sim \text{Uniform}(0.1, 1.0)$, $\gamma \sim \text{Uniform}(0.01, 0.5)$.
- **Population:** $N_{\text{pop}} = 1000$, with $S_0 = 990$, $I_0 = 10$, $R_0 = 0$.

2) *Discrete-Time Stochastic Simulator:* At each time step t :

$$\Delta I_t \sim \text{Binomial}\left(S_t, 1 - \exp\left(-\frac{\beta I_t}{N_{\text{pop}}}\right)\right), \quad (2)$$

$$\Delta R_t \sim \text{Binomial}(I_t, \gamma), \quad (3)$$

then

$$S_{t+1} = S_t - \Delta I_t, \quad I_{t+1} = I_t + \Delta I_t - \Delta R_t, \quad R_{t+1} = R_t + \Delta R_t, \quad (4)$$

3) *Two Simulation Modes:* Both modes use the same ε , the same σ_k , and the same observed summaries:

- 1) **Early exit:** stop when $I_t = 0$. Cheap particles finish early.
- 2) **Fixed horizon:** always run $T = 160$ steps.

Fixed T keeps the step count the same. Early exit does not. Having both on CPU means a later GPU timing is not stuck with one of those mixed in.

4) Summary Statistics and Distance:

$$s = (\text{peak infected count, time of peak, final recovered count}). \quad (5)$$

Distance is Euclidean after normalizing each component:

$$d(s_{\text{sim}}, s_{\text{obs}}) = \sqrt{\sum_{k=1}^3 \left(\frac{s_{\text{sim},k} - s_{\text{obs},k}}{\sigma_k} \right)^2}, \quad (6)$$

where σ_k comes from a pilot run and is then kept fixed.

5) How we fix SIR settings:

- 1) Simulate one observed trajectory from $(\beta, \gamma) = (0.4, 0.1)$ with a fixed seed and save s_{obs} .
- 2) Run a few thousand draws from the prior, get σ_k from those summaries.
- 3) Set ε to a quantile of the pilot distances so acceptance is about 1–5%.
- 4) Save s_{obs} , σ_k , and ε . Later CPU and GPU runs just read those files.

ε is left at the pilot quantile for the runs in the tables.

6) *What counts as correct for SIR:* (β, γ) is not well identified from three summaries, and there is no closed form posterior. We look at whether the samples sit nearer $(0.4, 0.1)$ than the prior, and we report mean and spread.

D. Metrics

For both workloads we report runtime, throughput (particles per second), number accepted, acceptance rate, and posterior summaries (mean and variance for Gaussian; means and spread for β and γ in SIR).

For SIR we also report steps per particle (mean, std, histogram), updates per second, and early exit vs fixed T at the same N .

IV. SEQUENTIAL IMPLEMENTATION

A. Architecture

Models and the sampler are separate. Each model is a struct with `sample_prior`, `simulate`, `distance`, and `observed_stats`. Kernels cannot use virtuals or most of the STL, so the CPU side is already written that way.

Accepted parameters are stored as one array per parameter, which is the layout we will want for coalesced loads later.

B. ABC Rejection Sampling Algorithm

Algorithm 1 is what both workloads run.

Algorithm 1 Sequential ABC rejection sampling.

Require: Model $\mathcal{M}$, particle count N , tolerance ε , random seed

Ensure: Accepted parameter samples, summary metrics

- 1: Initialize RNG with seed
 - 2: $s_{\text{obs}} \leftarrow \mathcal{M}.\text{observed_stats}()$
 - 3: `accepted` $\leftarrow$ empty list
 - 4: Start timer
 - 5: **for** $i = 1$ **to** N **do**
 - 6: $\theta_i \leftarrow \mathcal{M}.\text{sample_prior}(\text{rng})$
 - 7: $s_i \leftarrow \mathcal{M}.\text{simulate}(\theta_i, \text{rng})$
 - 8: $d_i \leftarrow \mathcal{M}.\text{distance}(s_i, s_{\text{obs}})$
 - 9: **if** $d_i \leq \varepsilon$ **then**
 - 10: Append θ_i to `accepted`
 - 11: **end if**
 - 12: **end for**
 - 13: Stop timer
 - 14: Compute and write summary statistics
-

C. Implementation Details

The code is C++17, compiled `-O2`. RNG is xoshiro256** seeded with SplitMix64. Normals are Box–Muller. Binomials use geometric waiting times. We store accepted parameters by column and record how many SIR steps each particle ran. Command line flags cover model, N , ε , seed, and SIR early exit. Timing runs start at seed 42; extra reps use 43, 44, and so on. This milestone is single threaded (no CUDA or OpenMP).

Compiler: Apple Clang 17, `-O2 -std=c++17`, macOS arm64. All timings are on that machine, one thread.

V. EXPERIMENTAL SETUP

A. Configuration

TABLE I
COMMON EXPERIMENT CONFIGURATION.

Parameter	Value
Particle counts	$10^4, 10^5, 10^6$
Execution model	Single-threaded CPU
Random seed	42
<i>Gaussian model</i>	
Observed value y_{obs}	1.0
Tolerance ε	0.1
<i>SIR model</i>	
Population N_{pop}	1000
Initial infected I_0	10
Time steps T	160
True parameters	$\beta = 0.4, \gamma = 0.1$
Tolerance ε	0.382 (2% pilot quantile)
SIR modes	Early-exit and fixed- T

B. Frozen SIR Calibration

σ_k and ε come from a pilot of a few thousand simulations. ε is a quantile of those distances (we wanted about 1–5% accepted), not something we tweaked by hand. s_{obs} , σ_k , and ε are saved in the repo and reused as is.

VI. CORRECTNESS VALIDATION

A. Gaussian Model

Eq. 1 is the ground truth. With $\varepsilon = 0.1$ the ABC mean and variance should sit near 0.5.

TABLE II
GAUSSIAN MODEL: POSTERIOR VALIDATION ($N = 10^6$).

Method	Mean	Variance
Analytical posterior	0.500	0.500
Sequential ABC	0.501	0.501

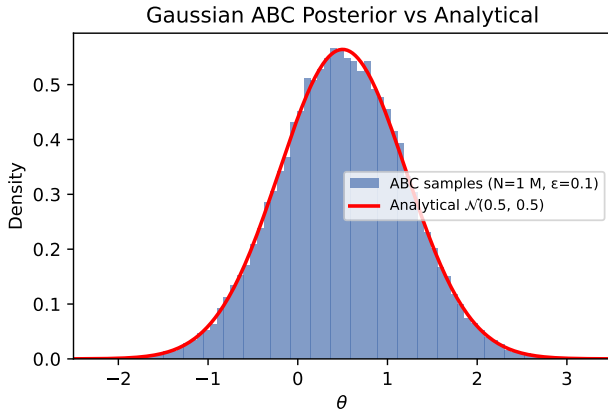


Fig. 1. Gaussian ABC posterior vs. analytical posterior ($N = 10^6$, $\varepsilon = 0.1$).

TABLE III
GAUSSIAN POSTERIOR VS. TOLERANCE ($N = 10^5$).

ε	Mean	Variance	Acceptance rate
1.0	0.420	0.570	42.2%
0.5	0.477	0.518	21.8%
0.1	0.489	0.504	4.4%
0.05	0.480	0.496	2.2%
Analytical	0.500	0.500	—

1) *Approximation Quality vs. Tolerance*: Smaller ε in Table III sits closer to $N(0.5, 0.5)$, with a lower acceptance rate.

B. SIR Model

SIR has no closed form posterior. We check that the ABC samples move from the prior toward $(0.4, 0.1)$. With only three summaries we do not expect a tight peak; we report means, standard deviations, and the scatter.

TABLE IV
SIR MODEL: POSTERIOR VALIDATION ($N = 10^6$, EARLY-EXIT MODE).

Parameter	True	Post. mean	Post. std
β	0.400	0.427	0.070
γ	0.100	0.117	0.022

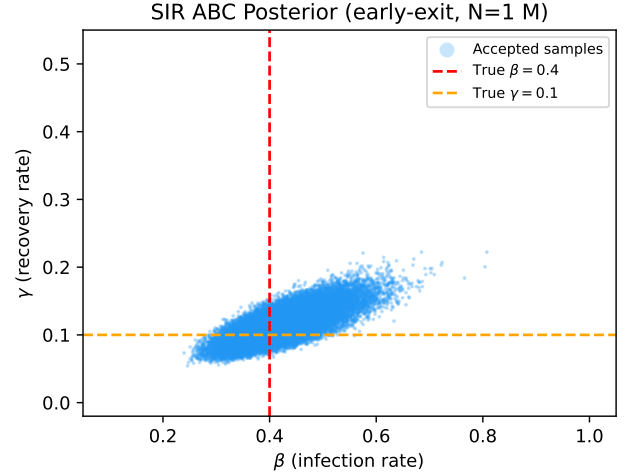


Fig. 2. SIR ABC posterior samples in (β, γ) space ($N = 10^6$, early-exit).

VII. SEQUENTIAL PERFORMANCE RESULTS

A. Gaussian Model

TABLE V
GAUSSIAN MODEL: SEQUENTIAL PERFORMANCE.

Particles	Runtime (s)	Throughput (p/s)	Accepted	Acc. rate
10^4	7.40×10^{-4}	13.5×10^6	443	4.43%
10^5	5.35×10^{-3}	18.7×10^6	4,390	4.39%
10^6	5.19×10^{-2}	19.3×10^6	43,907	4.39%

B. SIR Model

TABLE VI
SIR MODEL: SEQUENTIAL PERFORMANCE (EARLY-EXIT MODE).

Particles	Runtime (s)	Throughput (p/s)	Accepted	Acc. rate
10^4	0.206	4.85×10^4	225	2.25%
10^5	2.12	4.72×10^4	2,200	2.20%
10^6	21.15	4.73×10^4	22,033	2.20%

TABLE VII
SIR MODEL: SEQUENTIAL PERFORMANCE (FIXED- T MODE).

Particles	Runtime (s)	Throughput (p/s)	Accepted	Acc. rate
10^4	0.215	4.65×10^4	225	2.25%
10^5	2.15	4.65×10^4	2,200	2.20%
10^6	21.38	4.68×10^4	22,033	2.20%

C. SIR Work Metrics

TABLE VIII
SIR SIMULATION STEPS PER PARTICLE ($N = 10^6$).

Mode	Mean steps	Std. dev.
Early-exit	64.2	44.9
Fixed- T	160	0

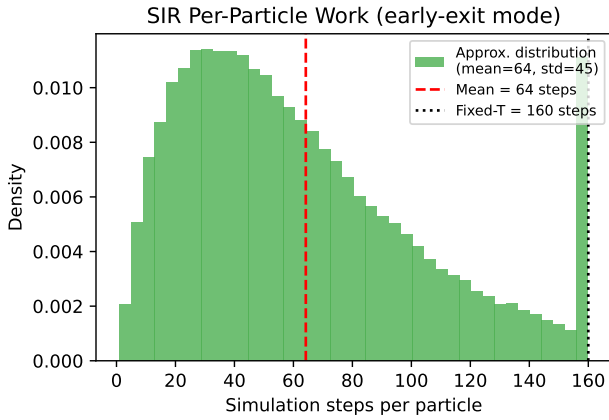


Fig. 3. Distribution of SIR simulation steps per particle (early-exit mode, $N = 10^6$).

Early exit has std 44.9 on steps. A warp on the GPU would wait on the longest trajectory in the group.

D. Workload Comparison

TABLE IX
PER-PARTICLE COST COMPARISON ($N = 10^6$).

Metric	Gaussian	SIR (early)	SIR (fixed)
Runtime (s)	5.19×10^{-2}	21.15	21.38
Throughput (p/s)	19.3×10^6	4.73×10^4	4.68×10^4
Updates/s	—	$64.2 \times 4.73 \times 10^4$	$160 \times 4.68 \times 10^4$

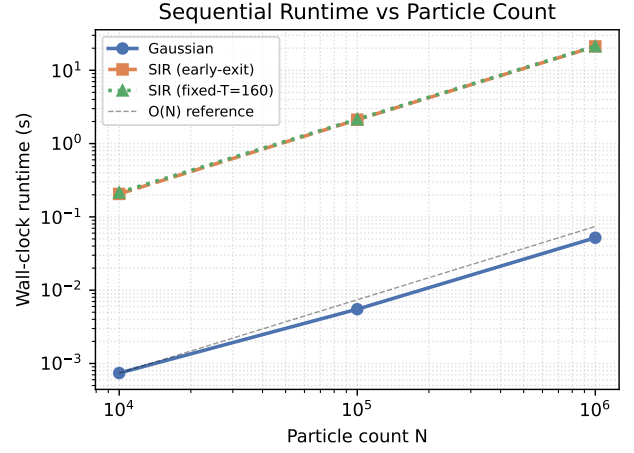


Fig. 4. Sequential runtime vs. particle count across all workloads (log-log scale).

VIII. CONCLUSION AND FUTURE WORK

The sequential sampler is in C++. Gaussian ABC at $\varepsilon = 0.1$ has mean and variance 0.501 against 0.5. SIR at $N = 10^6$ takes about 21 s, versus 0.05 s for Gaussian. Early exit averages 64 steps (std 45); fixed T is always 160. s_{obs} , σ_k , and ε are in results/ and stay as they are.

Next we implement one particle per CUDA thread with the same models and thresholds. Posterior mean/variance and acceptance rate are the check against this CPU run. Profiling can wait until that kernel exists.

REFERENCES

- [1] M. A. Beaumont, W. Zhang, and D. J. Balding, "Approximate Bayesian computation in population genetics," *Genetics*, vol. 162, no. 4, pp. 2025–2035, 2002.
- [2] S. A. Sisson, Y. Fan, and M. A. Beaumont, Eds., *Handbook of Approximate Bayesian Computation*. Chapman and Hall/CRC, 2018.
- [3] S. Kulkarni, M. M. Krell, S. Nabarro, and C. A. Moritz, "Hardware-accelerated simulation-based inference of stochastic epidemiology models for COVID-19," *ACM Journal on Emerging Technologies in Computing Systems*, vol. 18, no. 2, pp. 30:1–30:24, 2022.
- [4] R. Dutta, M. Schoengens, L. Pacchiardi, A. Ummadisingu, N. Widmer, P. Künzli, J.-P. Onnela, and A. Mira, "ABCpy: A high-performance computing perspective to approximate Bayesian computation," *Journal of Statistical Software*, vol. 100, no. 7, pp. 1–38, 2021.